# Real-Time PCB Defect Detection for Embedded Visual Inspection: A Same-Split Comparison of YOLO11s and RT-DETR-L

Aditya Varun Dhayapulay[1] and Paul Salvador Inventado[1]

California State University, Fullerton, Fullerton, CA, USA
adivd@csu.fullerton.edu; pinventado@fullerton.edu

**Abstract.** Printed circuit board inspection requires reliable localization of small defects at latency compatible with manufacturing workflows. We compare YOLO11s and RT-DETR-L on one 5,551/1,016/1,016 train, validation, and test split covering six PCB defect classes. The accepted practical configurations use YOLO11s at 1280 pixels and RT-DETR-L at 640 pixels. To separate architecture effects from input-resolution effects, the camera-ready study adds models trained and evaluated at a matched 640-pixel resolution under an identical batch-1, no-test-time-augmentation protocol on NVIDIA Tesla V100-SXM2-32GB hardware. At 640 pixels, YOLO11s obtains 0.852 recall and 0.482 mAP50–95, while RT-DETR-L obtains 0.839 recall and 0.462 mAP50–95. We also report RT-DETR-L at 1280 pixels, per-class results, and representative success and failure cases. The study contributes a reproducible, deployment-oriented benchmark rather than a new detector architecture. A browser-based inference demo is provided as an online deployment artifact; embedded-device and TensorRT benchmarking remain future work.

**Keywords:** PCB defect detection · embedded vision · YOLO11 · RT-DETR · object detection · industrial inspection

## 1 Introduction

Printed circuit boards (PCBs) are central to embedded and cyber-physical products. A missing hole, short, copper spur, or other visual defect on an unpopulated board can become substantially more expensive after assembly. Automated optical inspection is therefore not only an accuracy problem: a detector must find small defects, localize them well enough for review or repair, and execute within the timing and resource constraints of an inspection workflow.

This paper studies six defect classes: Missing_hole, Mouse_bite, Open_circuit, Short, Spur, and Spurious_copper. The accepted submission compared a completed YOLO11s model trained at 1280 pixels with a completed RT-DETR-L model trained at 640 pixels. Both checkpoints used the same dataset split and evaluator, but the input-size difference meant that the result was a comparison of practical completed configurations rather than a controlled architecture-only ablation.

The camera-ready study preserves that practical comparison and adds the following:

- a matched-resolution YOLO11s-640 versus RT-DETR-L-640 experiment;
- a completed RT-DETR-L high-resolution experiment at 1280 pixels and batch size 1, which did not require the planned 960-pixel fallback;
- a corrected canonical class-ID mapping for semantic per-class analysis;
- class-balanced detection examples and representative failure cases; and
- a complete manifest connecting dataset counts, software versions, hardware, seeds, checkpoints, tables, and figures.

The contribution is a traceable benchmark for choosing a practical PCB inspection baseline. We do not claim a new architecture or embedded-device performance.

## 2 Related Work

PCB defect inspection has progressed from reference-image comparison and rule-based machine vision to learned object detectors. Classical systems can be efficient when board registration, lighting, and product geometry are tightly controlled, but they often require product-specific thresholds and feature engineering. Learned detectors jointly classify and localize a defect, which is useful when the output feeds human review, repair guidance, or process analytics. The six-class PCB dataset introduced by Huang and Wei is a common reference for this problem [1].

YOLO-family detectors are widely used when latency matters because they predict classes and boxes in one forward pass. We use Ultralytics YOLO11s [6] as the compact convolutional baseline. RT-DETR [8] provides a contrasting transformer-style design intended for real-time end-to-end detection. The comparison is useful for PCB inspection because small, sparse defects stress both feature resolution and bounding-box precision.

Detector fusion can increase recall when missed defects are more costly than false alarms. The accepted study included Weighted Boxes Fusion (WBF) [4] as supporting analysis. The present resolution-control experiment focuses on single-model comparisons so that resolution and architecture effects remain interpretable.

## 3 Dataset and Experimental Protocol

### 3.1 Dataset

The primary corpus, denoted `YOLO_PCB`, combines the project PCB dataset with overlapping classes from DsPCBSD+ [3]. The deterministic preparation procedure uses seed 42, stratifies within source/class groups, and produces 5,551 training, 1,016 validation, and 1,016 test images. The validation partition contains 2,106 annotated instances and the test partition contains 2,179. The training split includes 800 class-balancing copy-paste images: 320 Missing_hole, 240 Mouse_bite, and 240 Spur examples.

A reproducibility audit found that an earlier dataset export changed the displayed class-name order without remapping label IDs. This does not change

**Table 1.** Canonical defect classes used by the controlled experiments.

| ID | Class |
|---|---|
| 0 | Missing_hole |
| 1 | Mouse_bite |
| 2 | Open_circuit |
| 3 | Short |
| 4 | Spur |
| 5 | Spurious_copper |

class-agnostic aggregate precision, recall, or mAP, but it can misname per-class rows. We therefore use two explicit dataset manifests: a legacy ID layout solely for reproducing the accepted aggregate baselines, and the canonical semantic mapping in Table 1 for all new training and per-class claims. Both manifests contain identical image assignments and instance counts.

### 3.2 Models and Training

The original YOLO11s checkpoint was trained for 50 epochs at 1280 pixels with batch size 12. The original RT-DETR-L checkpoint was trained for 10 epochs at 640 pixels with batch size 4. These checkpoints are preserved unchanged.

For the controlled study, YOLO11s is trained for 50 epochs at 640 pixels with batch size 12 and seed 42. RT-DETR-L is trained for 10 epochs at 640 pixels with batch size 4 and seed 42. The high-resolution RT-DETR-L run uses 10 epochs, batch size 1, and seed 42 at 1280 pixels. A one-epoch smoke test precedes every full training configuration. The 1280-pixel run completed successfully, so the planned 960-pixel out-of-memory fallback was not used.

### 3.3 Evaluation

Every reported model is evaluated with Ultralytics 8.4.51 and PyTorch 2.5.1 on a Tesla V100-SXM2-32GB. Evaluation uses batch size 1, zero dataloader workers, the same validation and test images, and no test-time augmentation. We report precision, recall, F1, mAP50, mAP50–95, and per-image preprocess, inference, postprocess, and total latency. Precision is the true-positive fraction among predictions, recall is the true-positive fraction among labeled instances, and F1 is their harmonic mean, consistent with the evaluator documentation used by the pipeline [5]. The mAP50–95 convention averages average precision over IoU thresholds from 0.50 to 0.95 in increments of 0.05, following the COCO evaluation convention [2].

Before new training begins, the archived checkpoints are evaluated on the rebuilt legacy split. The pipeline stops if precision, recall, mAP50, or mAP50–95 differs from the archived value by more than 0.001. The observed differences were approximately $10^{-8}$, confirming exact split and evaluator reproduction.

# 4 Results

## 4.1 Accepted Practical Configurations

Table 2 preserves the accepted comparison. It should be interpreted as a practical-configuration result: YOLO11s-1280 versus RT-DETR-L-640, not an input-size-controlled architecture comparison.

**Table 2.** Accepted practical configurations on the test split. All values are generated from the archived-checkpoint evaluation CSV.

| Model | P | R | F1 | mAP50 | mAP50–95 | Infer. ms | Total ms |
|---|---|---|---|---|---|---|---|
| YOLO11s 1280 | 0.880 | 0.866 | 0.873 | 0.902 | 0.502 | 12.8 | 15.2 |
| RT-DETR-L 640 | 0.886 | 0.839 | 0.862 | 0.887 | 0.470 | 48.8 | 49.8 |

The evaluation-only diagnostic changes the original YOLO11s checkpoint input size from 1280 to 640 without retraining. Its test recall falls from 0.866 to 0.810 and mAP50–95 falls from 0.502 to 0.453. This result is not the matched comparison; it demonstrates why simply resizing a checkpoint cannot replace resolution-specific training.

## 4.2 Matched Resolution and High-Resolution RT-DETR

Table 3 reports the controlled experiments. YOLO11s-640 and RT-DETR-L-640 use identical image assignments, resolution, evaluation batch size, evaluator, hardware class, and test-time augmentation setting. The matched recall difference (YOLO11s minus RT-DETR-L) is 0.013, and the matched mAP50–95 difference is 0.020.

**Table 3.** Resolution-controlled and high-resolution configurations on the test split. Values are generated from the saved evaluation CSV.

| Model | P | R | F1 | mAP50 | mAP50–95 | Infer. ms | Total ms |
|---|---|---|---|---|---|---|---|
| RT-DETR-L 640 | 0.867 | 0.839 | 0.853 | 0.881 | 0.462 | 48.0 | 49.0 |
| RT-DETR-L 1280 | 0.766 | 0.534 | 0.630 | 0.604 | 0.295 | 72.4 | 74.4 |
| YOLO11s 640 | 0.870 | 0.852 | 0.861 | 0.892 | 0.482 | 11.4 | 13.2 |

The high-resolution RT-DETR-L row is reported separately because it changes both input size and computational cost. It answers whether the transformer model benefits from additional spatial detail, but it is not a replacement for the matched 640-pixel comparison. In this run, increasing RT-DETR-L to 1280 pixels did not improve the measured configuration: test mAP50–95 changed from 0.462 at 640 pixels to 0.295, while total latency increased from 49.0 to 74.4 ms. This result suggests that additional pixels alone do not compensate for the fixed 10-epoch schedule and substantially higher computational cost.

### 4.3 Per-Class Behavior

Table 4 uses only models trained with the corrected canonical label mapping. It therefore replaces semantic per-class claims from the accepted draft while leaving the accepted aggregate baseline unchanged.

**Table 4.** Matched 640-pixel per-class test results using canonical labels.

| Class | Model | P | R | mAP50 | mAP50–95 |
|---|---|---|---|---|---|
| Missing_hole | YOLO11s 640 | 0.990 | 0.986 | 0.994 | 0.569 |
| Missing_hole | RT-DETR-L 640 | 0.962 | 1.000 | 0.995 | 0.567 |
| Mouse_bite | YOLO11s 640 | 0.821 | 0.763 | 0.835 | 0.403 |
| Mouse_bite | RT-DETR-L 640 | 0.818 | 0.765 | 0.829 | 0.403 |
| Open_circuit | YOLO11s 640 | 0.848 | 0.850 | 0.890 | 0.525 |
| Open_circuit | RT-DETR-L 640 | 0.882 | 0.880 | 0.917 | 0.513 |
| Short | YOLO11s 640 | 0.880 | 0.892 | 0.942 | 0.551 |
| Short | RT-DETR-L 640 | 0.881 | 0.865 | 0.900 | 0.505 |
| Spur | YOLO11s 640 | 0.869 | 0.767 | 0.839 | 0.381 |
| Spur | RT-DETR-L 640 | 0.818 | 0.729 | 0.794 | 0.342 |
| Spurious_copper | YOLO11s 640 | 0.815 | 0.853 | 0.852 | 0.462 |
| Spurious_copper | RT-DETR-L 640 | 0.844 | 0.795 | 0.852 | 0.443 |

### 4.4 Representative Predictions

The prediction audit matches detections to ground truth at IoU 0.50. Figure 1 selects one successful example per class where available. Figure 2 ranks examples by a miss-weighted error score, $5FN+FP$, and favors class diversity. The selection manifest records every source image, ground-truth count, prediction count, true positive, false positive, and false negative.

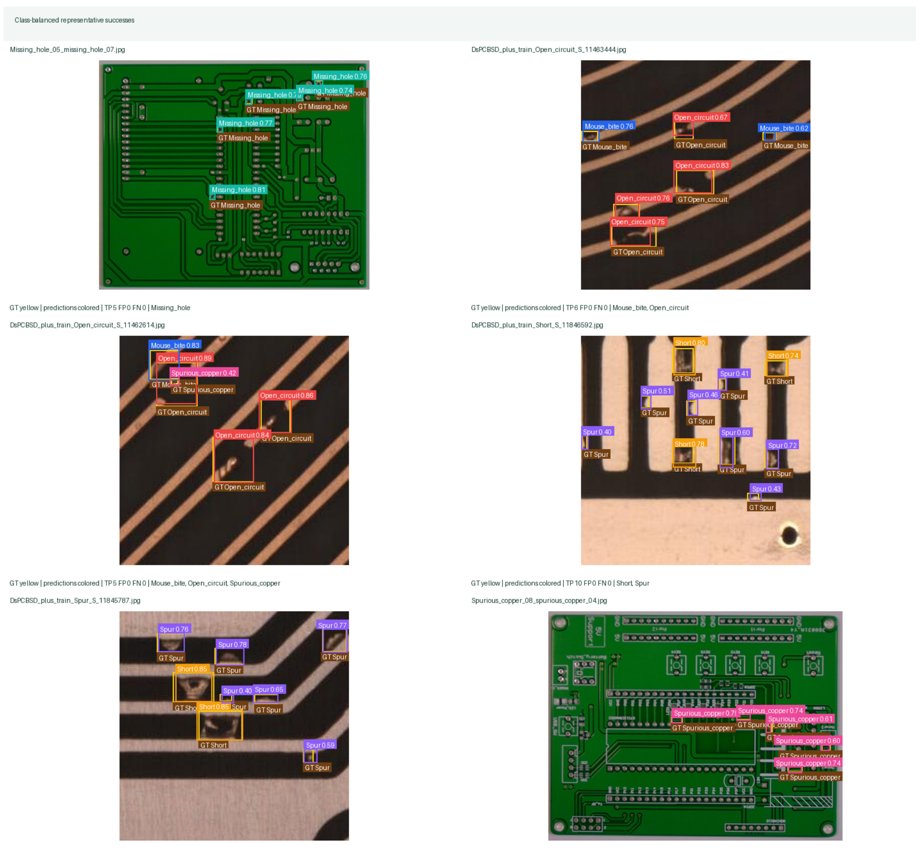


**Fig. 1.** Class-balanced representative detections from the matched YOLO11s-640 test run.

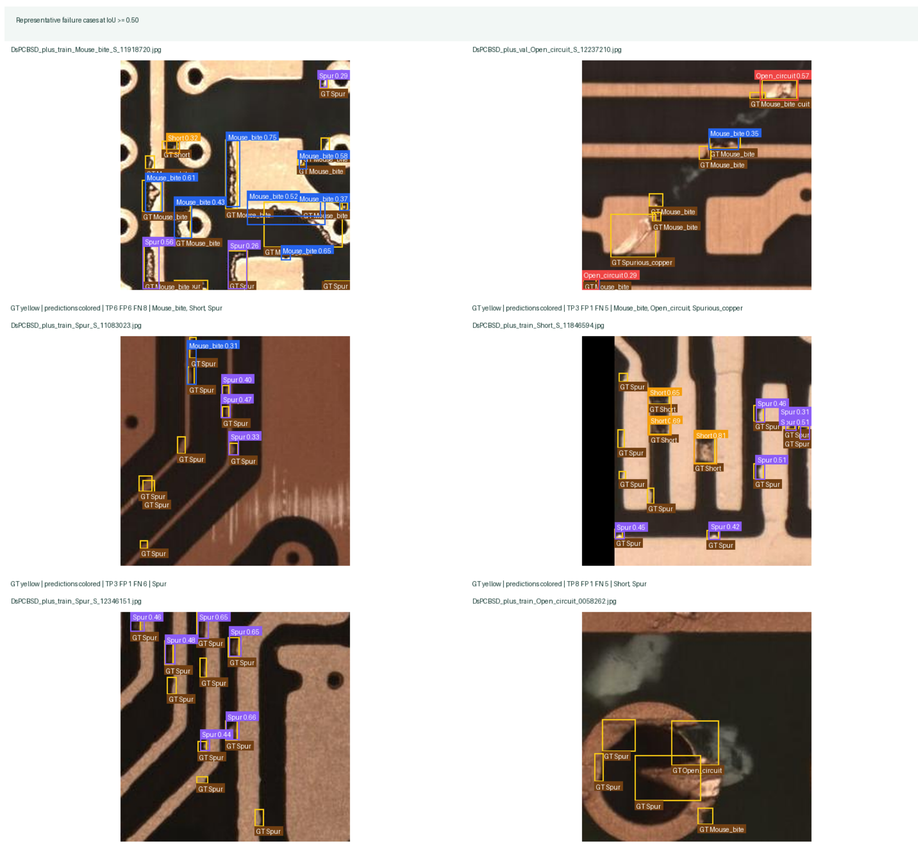


**Fig. 2.** Representative failure cases selected by false-negative and false-positive counts at IoU 0.50.

## 5 Discussion

The original practical comparison favored YOLO11s-1280 on recall, strict localization, and latency while RT-DETR-L-640 had a small precision advantage. Because the configurations used different input sizes, those results support a deployment choice among completed models, not a clean architectural ranking.

The matched 640-pixel experiment directly addresses that limitation. Its interpretation should focus on the direction and magnitude of the precision, recall, mAP, and latency differences under controlled input resolution. The high-resolution RT-DETR-L experiment then shows whether additional spatial detail closes any localization gap and what latency cost accompanies it. Together, the two tables distinguish resolution effects from architecture effects more clearly than either experiment alone.

The matched-resolution study is not a resource-normalized architecture benchmark. It preserves the completed per-model training schedules: 50 epochs and batch size 12 for YOLO11s, compared with 10 epochs and batch size 4 for RT-DETR-L. The models also differ in parameter count, optimization behavior, and computational cost. A claim about detector architecture alone would require equalized training compute, latency, memory, FLOPs, or a broader hyperparameter study. We therefore limit the conclusion to the measured configurations.

The gap between mAP50 and mAP50–95 remains important for inspection. A model may identify the correct defect region at IoU 0.50 while still placing a loose box at stricter thresholds. Better localization may require high-resolution crops, label review, class-specific augmentation, or a second-stage refinement model rather than only a larger detector.

## 6 Deployment Scope and Limitations

The trained YOLO checkpoint is served through a public Hugging Face Space at `https://adiivd-pcb-defect-detection.hf.space`. The browser workflow accepts an uploaded PCB image and returns annotated detections, class counts, confidence values, and downloadable results. This is an online deployment artifact and usability demonstration, not an embedded-device benchmark.

All latency values in this paper are V100 measurements. We have not measured either detector on Jetson, TensorRT, RK3588, FPGA, or another edge-oriented target. We therefore make no claim about target-device latency, throughput, energy, or memory. The dataset is also an in-distribution benchmark rather than a factory-domain generalization study. Lighting variation, registration error, product changes, and real manufacturing noise require separate validation.

## 7 Conclusion

This work preserves the accepted YOLO11s-1280 versus RT-DETR-L-640 practical comparison and adds the matched-resolution evidence requested by reviewers. The controlled pipeline uses one split, canonical labels, pinned V100 hardware, batch-1 evaluation, no TTA, saved CSV tables, per-class metrics, prediction figures, and an auditable manifest. The results support a more careful distinction between architecture and resolution effects while retaining the deployment-oriented goal of the original study. Future work will evaluate real factory captures, resource-normalized training, high-resolution crops, and actual edge-device deployment. Jetson and TensorRT remain future work until they can be measured directly.

## Acknowledgments

We thank Christopher Ryu for support with Nautilus/NRP access and research computing resources. This work used resources available through the National Research Platform at the University of California, San Diego [7]. Cursor and OpenAI Codex were used as programming assistants for code navigation, reproducibility checks, and manuscript proofreading. The authors reviewed all suggestions and remain responsible for the experiments, analysis, code, figures, and text.

## References

1. Huang, W., Wei, P.: A pcb dataset for defects detection and classification. arXiv preprint arXiv:1901.08204 (2019)
2. Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft coco: Common objects in context. In: Computer Vision – ECCV 2014. Lecture Notes in Computer Science, vol. 8693, pp. 740–755 (2014). https://doi.org/10.1007/978-3-319-10602-1_48
3. Lv, S., Ouyang, B., Deng, Z., Liang, T., Jiang, S., Zhang, K., Chen, J., Li, Z.: A dataset for deep learning based detection of printed circuit board surface defect. Scientific Data **11**, 811 (2024). https://doi.org/10.1038/s41597-024-03656-8, dataset: https://doi.org/10.6084/m9.figshare.24970329.v1
4. Solovyev, R., Wang, W., Gabruseva, T.: Weighted boxes fusion: Ensembling boxes from different object detection models. Image and Vision Computing **107**, 104117 (2021). https://doi.org/10.1016/j.imavis.2021.104117
5. Ultralytics: Performance metrics deep dive. https://docs.ultralytics.com/guides/yolo-performance-metrics/ (2026), accessed June 2026
6. Ultralytics: Yolo11 models. https://docs.ultralytics.com/models/yolo11/ (2026), accessed May 2026
7. Weitzel, D., Graves, A., Albin, S., Zhu, H., Würthwein, F., Tatineni, M., Mishin, D., Khoda, E.E., Sada, M.F., Smarr, L., DeFanti, T., Graham, J.: The national research platform: Stretched, multi-tenant, scientific kubernetes cluster. In: Practice and Experience in Advanced Research Computing 2025: The Power of Collaboration. pp. 1–5 (2025). https://doi.org/10.1145/3708035.3736060
8. Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., Chen, J.: Detrs beat yolos on real-time object detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 16965–16974 (2024). https://doi.org/10.1109/CVPR52733.2024.01605